ShieldedETH: An Immutable Multi-Asset Shielded Payment Pool for Ethereum

Anonymous

2026-10-03

Abstract

ShieldedETH is a research implementation of an immutable, multi-asset shielded payment pool for Ethereum L1, with no administrative key or protocol percentage fee. Deposits bind a hiding partial commitment, amount and asset into a SHA-256 queue accumulator without requiring a proof or on-chain Poseidon evaluation. Permissionless insertion proofs advance a depth-32 note tree by up to sixteen entries. At low volume, a direct spend can absorb one pending deposit and insert its own outputs. Independent transfer proofs share verification through a randomized Groth16 batch equation using n+3 pairings rather than 4n. Pure transfers hide their asset in public contract fields, subject to recipient, broadcaster and traffic-analysis leakage. We implement the v0.8 contracts, circuits, reference model and adversarial tests, and measure full signed transactions and actual full-depth proofs in a local Cancun EVM. Native and hashed statement profiles are compared using generated benchmark evidence. The evaluated Groth16 backend requires trusted setup and uses development-only keys. It is a research baseline that does not meet the production requirement of no trusted setup. No transparent replacement, production deployment or external audit is claimed.

1 Introduction

Account-based blockchains publish every transfer's sender, recipient and amount. Shielded-note systems, introduced by Zerocash [1] and refined in Zcash [2], replace balances with hiding commitments to notes, mark spends with nullifiers, and prove conservation in zero knowledge. Deployments on Ethereum face constraints that a dedicated chain does not: every hash, storage write and pairing is metered, the contract code size is bounded [3], and any administrative key or upgrade path is a standing point of control over user funds.

Existing Ethereum systems resolve these constraints differently. Tornado Cash Classic [4] uses fixed denominations and no internal transfers. Tornado Nova [5] supports arbitrary amounts and internal transfers, but runs on a separate chain behind a bridge and is upgradeable by governance. RAILGUN [6] supports many assets on L1, but charges a 0.25% fee on shielding and on unshielding and is governed. Rollups such as Aztec [7] generalize to private programmability under a different trust and settlement model.

This paper presents ShieldedETH, a payment pool with three fixed properties: immutable contracts with no owner, pause or upgrade function; no percentage or deployer fee; and settlement on Ethereum L1. Within those constraints, the design targets execution cost and low-volume usability. The contributions are:

1. Deposits without proofs or on-chain Poseidon. The contract records the deposited amount, the asset and a hiding partial commitment; the final note commitment is computed later, inside the proof that adds the note to the tree (Section 4.3).
2. Tree updates moved out of the contract. New notes wait in a queue that costs one SHA-256 call per transaction. Any party can add a batch of queued notes to the tree with one proof, and a fixed schedule guarantees that a valid next batch always exists. When nothing else is waiting, a spend updates the tree itself (direct mode), including a deposit made moments earlier that the same proof spends (Section 4.1, Section 4.4, Section 4.5).
3. Hidden-asset notes. One constraint makes the asset public only when value leaves the pool (Section 4.6).
4. Batch-verified transfers. A randomized Groth16 batch equation checks n independent user proofs with $n+3$ pairings. A user's proof is valid whether submitted alone or in a batch (Section 4.7). A public-input hashing profile is analyzed as a per-circuit alternative (Section 4.8).
5. Implemented relations and reproducible evidence. Actual depth-32 Groth16 proofs run through the pool, with an independent plaintext liability ledger, adversarial rejection tests, complete transaction receipts and desktop proving measurements (Section 5, Section 6).

The production design additionally requires no trusted setup. The evaluated Groth16 implementation does not meet this requirement; its measurements establish a baseline for evaluating a future transparent replacement, not an eligible production backend.

The design adapts the note model and leakage analysis of Shielded Bitcoin [8]. That protocol derives state by replaying Bitcoin transactions because Bitcoin cannot verify proofs; on Ethereum, the contract verifies proofs directly and stores the authenticated state.

2 Background and Related Work

Shielded notes. A note is an opening $(v, a, ...)$ of a hiding, binding commitment c held in an append-only Merkle tree. Spending proves knowledge of an opening under a recent root, a spend key, and a nullifier derived deterministically from the note; the verifier records the nullifier to prevent double spends. Zcash Sapling binds the nullifier to the note's tree position so that duplicate commitments cannot suppress each other ("Faerie Gold") [2]. ShieldedETH uses the same rule.

Ethereum mixers and pools. Tornado Classic deposits insert a MiMC commitment into an on-chain Merkle tree, and withdrawals prove membership [4]. Nova adds arbitrary amounts, change outputs and internal transfers [5]. Both hash the Merkle path in the contract on every insertion, so tree maintenance dominates deposit cost. RAILGUN supports ERC-20, ERC-721 and ERC-1155 assets and a network of paid broadcasters [6]. Privacy Pools [9] add association-set proofs intended to support regulatory compliance; compliance mechanisms are outside the scope of this paper.

Proof aggregation. Recursive composition and inner-product aggregation such as SnarkPack [10] reduce many proofs to one, but they require a different proof system for the outer proof or a large verifier. Small-exponent batch verification [11], [12] instead combines verification equations with random coefficients. For Groth16 [13], one pairing per proof remains, but the fixed pairings and the public-input multi-scalar multiplication are shared. ShieldedETH applies this test on-chain to unmodified user proofs.

Deferred insertion. Keeping the tree out of contract execution and advancing its root by proof is common in rollups and identity registries. ShieldedETH combines it with proofless deposits, a schedule that guarantees progress, and an idle-queue path that needs no separate batch proof at low volume.

3 Model and Goals

Parties. Users hold a seed from which spend, view and recovery keys derive. Optional broadcasters and aggregators submit users' transactions and are paid by a shielded output. Batchers submit insertion proofs and earn public ETH tips. Indexers serve public data. None of these parties receives spend or view keys.

Adversary. The adversary controls any number of users, broadcasters, batchers and indexers; observes the chain and mempool; front-runs and reorders transactions; submits malformed inputs; and deploys arbitrary contracts. It cannot break SHA-256 or Poseidon collision resistance, the knowledge soundness of Groth16 over BN254, or the discrete-logarithm assumptions of ristretto255.

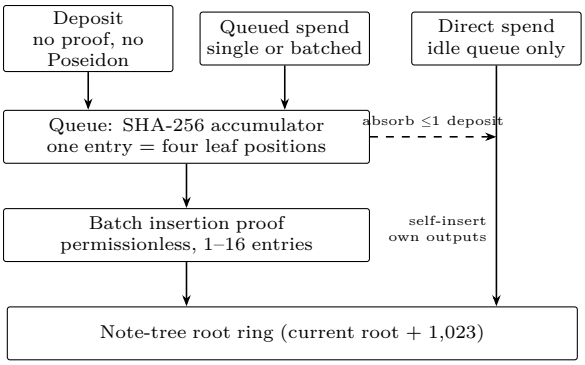


Figure 1: State flow. Deposits and queued spends append entries to a SHA-256 accumulator, and insertion proofs advance the note-tree root. A direct spend absorbs at most one pending deposit and inserts its own outputs in the same call. All spends record their nullifiers immediately.

Goals. (G1) Balance: for every asset, total spendable note value never exceeds the deposits of that asset minus its withdrawals. (G2) Spend authority: only the holder of a note's spend key can nullify it. (G3) Transaction privacy: a pure transfer hides input membership, amounts and asset from an observer of contract fields, subject to the leakage and observer capabilities in Section 7. (G4) Liveness: any pending entry can be inserted by any party willing to pay gas. (G5) No privileged control: no key can alter rules, pause operations or move backing.

Non-goals. The design does not guarantee ciphertext delivery (a sender can burn value by encrypting badly), hide deposit and withdrawal amounts, or provide cross-asset transfers inside one proof.

4 Design

4.1 Note-tree maintenance

A note can be spent only after its commitment is a leaf of the Merkle tree whose root the contract stores. Adding a leaf changes every node on the path from that leaf to the root: 32 Poseidon evaluations at depth 32. Tornado Classic and Nova recompute this path inside the contract on every deposit or transaction, which adds tree-hashing work to those operations. We do not benchmark those systems in this evaluation. ShieldedETH keeps the contract out of that computation and has it check proofs of tree updates instead.

The contract keeps three pieces of state for this purpose: a queue of entries not yet in the tree, the current root, and a ring of recent roots that spends may reference. Each deposit or transfer appends one entry to the queue, which

costs one SHA-256 call and a few storage writes. The root then advances in one of two ways (Figure 1):

- Batch insertion. Any party, typically a batcher paid by tips, takes the next 1–16 queued entries and submits one proof stating "appending these entries to the tree with root R yields root R'". The contract checks that the entries are exactly the next ones in its queue and that the proof is valid, then stores R'. One proof covers up to 16 entries.
- Direct (self-) insertion. If no other entries are waiting (apart from at most one deposit and any exits), a spender's own proof also computes the root obtained by appending the spender's outputs, and the waiting deposit if present. The transaction then updates the tree itself, and no batcher or second transaction is involved.

Queued notes become spendable once a batch has inserted them; notes created in direct mode are spendable immediately. Nullifiers are recorded in both cases when the spend executes, so double spends are rejected regardless of insertion timing.

4.2 Notes, keys and delivery

All note hashing uses Poseidon [14] with the domain tag in the capacity element: $\mathsf{P}_t(x_1,\dots,x_m)$ is the first output of the width-$(m{+}1)$ permutation initialized with t. From a spend scalar sk, the wallet derives $ak = \mathsf{P}_{AK}(sk)$ and $nk = \mathsf{P}_{NK}(sk)$. A diversified address with 11-byte diversifier d contains $\mathrm{own}_d = \mathsf{P}_{OWN}(ak,d)$ and a ristretto255 [15] key $pk_d = [ivk]\,H_d$, where H_d hashes d to the group. A note of value $v < 2^{128}$ in asset a with randomness ρ is committed as

$$p = \mathsf{P}_{PART}(\mathrm{own}_d, \rho),$$
$$c = \mathsf{P}_{CM}(p, v, a).$$

The nullifier of the note at leaf position π is $\mathrm{nf} = \mathsf{P}_{NF}(\delta, nk, c, \pi)$, where δ is an immutable 253-bit pool domain. Binding π gives each funded commitment occurrence a distinct nullifier.

Each output carries a 109-byte ciphertext: an ephemeral key $epk = [esk]H_d$ with esk derived from ρ, followed by ChaCha20-Poly1305 [16] under an HKDF [17] key over $[esk]pk_d$. A recipient detects notes with one scalar multiplication per output, and accepts a note only if it reconstructs c. A 400-byte batch encrypted under the sender's outgoing key allows seed-only recovery of sent notes.

4.3 Proofless deposits

A deposit supplies (a, p, v, ct). The contract checks that p is a canonical nonzero field element and that the asset is on the allowlist, collects exactly v units, and appends a queue record whose words are $(p, v, a, 0)$. No proof is verified and no Poseidon is evaluated on-chain. The insertion circuit computes the leaf $c = \mathsf{P}_{CM}(p, v, a)$ from the record words. Because the record is bound by the accumulator (Section 4.4), a prover can insert no leaf other than the one the contract accepted. A depositor who publishes a p without a valid opening loses those funds; no depositor can create a note worth more than v, or a note in a different asset.

For token deposits, the contract checks that the pool balance rose by exactly v and the sender's balance fell by exactly v, which rejects fee-on-transfer and short-delivering tokens. `depositMany` accepts up to 64 items, pulls each asset's total once, and emits records byte-identical to single deposits.

4.4 Queue, accumulator and proved insertion

Each deposit, transfer and exit appends one entry e that owns leaf positions $4e,\dots,4e{+}3$ of a depth-32 tree. With $\mathsf{H}_{253}(x) = x \bmod 2^{253}$ applied to a 32-byte digest,

$$\mathrm{acc}_{e+1} = \mathsf{H}_{253}(\mathrm{SHA256}(\mathrm{acc}_e \,\|\, \kappa \,\|\, w_0 \| w_1 \| w_2 \| w_3 \,\|\, h_{\mathrm{ext}} \,\|\, \tau)),$$

where κ is the entry kind, w_i are record words, h_{ext} is the hash of the canonical transaction body, and τ is the insertion tip. Each record is 209 bytes (four SHA-256 blocks), and the SHA-256 precompile makes a chain step cost a few hundred gas.

An insertion proof for $k \le 16$ consecutive entries has one public input, the hash of the statement

$$(\mathrm{pool}, \mathrm{chain}, \mathrm{circuit}, R_{\mathrm{old}}, R_{\mathrm{new}}, \mathrm{start}, k, \mathrm{acc}_{\mathrm{start}}, \mathrm{acc}_{\mathrm{end}}, \textstyle\sum \tau, \beta).$$

The circuit recomputes the record chain and derives each entry's four leaves (completing deposit commitments). It proves that each depth-2 slot was empty under the running root and replaces it along the same path. The beneficiary β is inside the hash, so a copied proof cannot redirect the tips.

Guaranteed progress. The contract retains the accumulator at the inserted prefix, at every multiple of 16, and for the last 1,024 counts. For inserted prefix s and queue length $n > s$, the step to $\min(n, 16\lfloor s/16 \rfloor + 16)$ always has both endpoints retained. Any backlog therefore drains in valid batches of 1–16 entries.

4.5 Direct mode

At low volume, a dedicated batch proof costs about as much as the transfer it inserts, and nobody may be running a batcher. Direct mode applies when the pending prefix consists only of exits (zero subtrees, which leave the root unchanged) and at most one deposit. The caller supplies the pending records. The contract re-hashes them

from the stored inserted-prefix accumulator to the queue head, and passes the deposit tuple $(i_{\text{abs}}, p, v, a)$, or zeros, as public inputs. The circuit then:

(i) proves slot i_{abs} empty under the current root R_0 and inserts the deposit subtree, giving R_1;
(ii) proves every spent input's membership under R_1;
(iii) proves the own slot empty under R_1 and inserts the four output commitments, giving the public root R_2.

Checking membership under R_1 is what lets a user spend a deposit that was still waiting in the queue: the same proof first adds the deposit to the tree and then spends it, in one call and without a batcher. The reward beneficiary and the expected queue state are part of the hashed transaction body, so a front-runner cannot reassign the absorbed deposit's tip. A stale call reverts before proof verification.

4.6 Spend relation and hidden assets

A transfer consumes two (or eight) inputs and creates four outputs. Every enabled input and every output commits to the same private asset a. With public withdrawal W and $\mathsf{publicAmount} = 0$ if $W = 0$, else $r - W$ (the field modulus), the circuit enforces

$$\sum v_{\text{in}} + \mathsf{publicAmount} = \sum v_{\text{out}},$$
$$\mathsf{publicAmount} \cdot (a - \mathsf{publicAsset}) = 0.$$

The second equation forces $\mathsf{publicAsset} = a$ whenever value leaves the pool. For $W = 0$ the contract requires $\mathsf{publicAsset} = 0$, so a pure transfer omits its asset from that public field. Observer-dependent leakage remains as discussed in Section 7. All values are range-checked below 2^{128}, so the field equation is integer conservation. Disabled input slots publish a dummy nullifier $\mathsf{P}_{DUM}(\delta, \text{seed}, j)$ that is indistinguishable from a real one. Insertion tips are paid in ETH by the submitter and never drawn from note value, because drawing them from the note would make $\mathsf{publicAmount}$ nonzero and reveal a.

Liabilities are tracked per asset. Boundary operations check ETH backing for ETH notes, tips and credits, and check the selected token's backing when applicable. Observable short transfers and surcharges revert atomically. Tests show that a paused token's failed payout restores the spend and does not prevent an unrelated ETH exit. These checks cannot establish honest token balance reporting or prevent issuer confiscation, rebasing, depegging or malicious upgrades. An asset-failure isolation claim is conditional on the admitted token behavior, not a guarantee for arbitrary contracts.

Asset identifiers. The label a is a field element, and the contract decides which labels exist. The evaluated profile maps labels $0, \ldots, 7$ to a list of assets fixed at deployment (native ETH and ERC-20 tokens), each with its own deposit cap. A proposed permissionless profile would use the token address itself as the label and require different asset-range constraints and liability indexing. Exact transfer checks alone would not establish compatibility with arbitrary ERC-20 behavior. A pure transfer still omits its asset from public contract fields; this does not guarantee statistical indistinguishability across assets. The permissionless profile is an unimplemented proposal, outside the evaluated v0.8 allowlist profile.

4.7 Batch verification of independent transfers

Groth16 [13] accepts (A, B, C) for public inputs $x \in \mathbb{F}_r^k$ if

$$e(-A, B)\, e(\alpha, \beta)\, e(L(x), \gamma)\, e(C, \delta) = 1,$$

with $L(x) = \text{IC}_0 + \sum_j x_j \text{IC}_j$. For n proofs of one circuit, the verifier derives $r_0 = 1$ and $r_i = \text{Keccak}(\sigma, i) \bmod r$ for $i \geq 1$, from a seed σ that hashes the chain identifier, the verifier address, all proofs and all inputs [18]. It accepts if

$$\prod_i e(-r_i A_i, B_i)\; e(S\alpha, \beta)\; e(S\,\text{IC}_0 + \textstyle\sum_j T_j \text{IC}_j, \gamma) \cdot e(\sum_i r_i C_i, \delta) = 1,$$

with $S = \sum_i r_i$ and $T_j = \sum_i r_i x_{ij}$. The check uses $n+3$ pairings and $2n+k+2$ scalar multiplications, against $4n$ pairings and nk scalar multiplications for separate checks.

Soundness. Let $X_i \in \mathbb{G}_T$ be the left side of proof i's equation. The batch computes $\prod_i X_i^{r_i}$. If some $X_j \neq 1$ with $j \geq 1$, then for any fixed choice of the other coefficients at most one value of r_j makes the product 1, so a uniform r_j gives acceptance probability at most $1/r$. If only $X_0 \neq 1$, the product differs from 1 for every choice. Modeling Keccak as a random oracle, an adversary making q oracle queries succeeds with probability at most about $q/r < 2^{-190}$ for $q = 2^{64}$. The argument needs every B_i in the order-r subgroup of $\mathbb{G}_2$ and every A_i, C_i on the curve, both enforced by the precompiles [19].

Aggregator lane. `transactBatch` accepts up to 16 queued pure transfers ($W = 0$). It validates each entry statically; any malformation reverts the call. It then marks as stale the entries that other transactions have invalidated: an expired deadline, an evicted root, an already-spent nullifier, or a nullifier used by an earlier included entry. Stale entries are skipped and their tips credited to the submitter. The remaining entries are verified with one `verifyBatch` call and applied in order.

Each entry's canonical body is exactly what a single submission would hash, so a user's proof is lane-agnostic: whichever submission lands first succeeds, and the other is skipped or reverts. An aggregator is an ordinary broadcaster paid by each user's shielded fee output; it gains no keys and no pool privilege.

Table 1: Compiled depth-32 R1CS constraints (Circom 2.1.9, O2). Hashed spend circuits each have one public input.

Circuit	Native k	Native	Hashed
Transfer 2→4	12	21,522	238,587
Transfer 8→4	18	77,172	388,293
Exit 2→0	8	18,994	173,355
Direct 2→4	18	52,265	363,386
Insertion, max 1	1	320,128	–
Insertion, max 16	1	2,396,158	–

Table 2: Total transaction gas. Multi-item rows divide one receipt by the stated number of accepted items.

Operation	Native	Hashed
First token deposit	213,303	213,279
20 deposits, per item	48,777	48,775
Insert 16 entries	292,551	292,539
Queued transfer	471,640	386,411
Batch, per included transfer	239,535	223,191
Direct token spend	588,841	456,607
Token withdrawal + change	490,132	404,915
Two-note token exit	436,900	383,014
ETH exit	405,053	351,143
Eight-input consolidation	651,335	519,050

4.8 Statement profiles

A proof's public inputs cost about 6,150 gas each for the elliptic-curve work alone under EIP-1108 [20]. The hashed profile replaces a circuit's k inputs with $\mathsf{H}_{253}(\mathrm{SHA256}(x_1\|\cdots\|x_k))$. It decomposes each word with an alias-checked gadget, so the hashed bytes are the canonical integer below r. The profile is an immutable per-circuit deployment flag, and the constructor checks every verifier's arity against it. Section 6 shows that hashing pays off mainly for single submissions, because batch verification already shares the input work across the batch.

5 Implementation

The canonical implementation lives in `contracts/`, `circuits/` and `ref/`. The v0.7 documents provide inherited design context; v0.8 code and its generated evidence define the evaluated candidate. Circuits use Circom 2.1.9, circomlib 2.0.5 and O2 optimization. Groth16 proofs use BN254 via snarkjs 0.7.5. Contracts target Solidity 0.8.26, via-IR and Cancun, including an EIP-1153 reentrancy lock [21].

Verifier and pool. One generic verifier implementation serves each circuit, with a separate deployment for each verifying key. The IC points occupy an immutable data contract; single and batch verification check canonical inputs and proof coordinates before calling the BN254 precompiles. The pool checks each verifier's public-input arity against the immutable statement profile.

The pool runtime is 24,399 bytes, 177 below EIP-170. The generic verifier is 5,139 bytes and the E2 kernel is 8,140 bytes. All use Solidity 0.8.26, via-IR, optimizer runs 10, and Cancun. These are compiled runtime-template sizes; deployed addresses and immutable values vary.

Setup and production eligibility. Groth16 requires structured reference parameters and a circuit-specific phase-two setup. Tests use public PSE phase-one parameters and uncontributed phase-two development keys. The downloaded parameter digests are pinned for reproducibility; this work has not independently verified the ceremony transcript. These keys are unsuitable for accepting funds. The production requirement, recorded in the current v0.8 specification, prohibits soundness depending on destruction of secret setup material at any proof layer. Public, reproducible parameter derivation is permitted. A ceremony or trusted-setup wrapper cannot make this baseline eligible for release. A transparent replacement must preserve the payment relations and receive independent review and complete transaction benchmarks.

Tests. Witness tests assert both acceptance and rejection, and treat missing artifacts or checker failures as infrastructure errors. Actual relation proofs cover deposits, direct first-note spends, queued transfers, full and partial insertion, public withdrawals, exits and eight-input consolidation in both statement profiles. Contract tests compare liabilities with an independent plaintext note ledger and compare queue records, accumulators and roots with the reference model. Rejection tests cover malformed proofs, altered bodies, stale submissions, token transfer failures, recipient refusal, credit rollback, reentrancy and integer boundaries. These finite tests are evidence of tested behavior, not a proof of security.

6 Evaluation

The reference run used Apple M5 (arm64), 10 logical CPUs, 32 GiB RAM, Node v24.19.0, and snarkjs 0.7.5. All ten full-depth circuit variants generated proofs accepted independently by snarkjs and by the contracts. The maximum insertion relation was exercised with 16, 4, and 14 active entries.

Gas is the complete transaction charge reported by VM.runTx under Cancun rules, including intrinsic calldata cost and applied refunds. Every call is a separately signed transaction with a fresh access list and original-storage accounting. The sequence begins from a newly deployed pool, then exercises deposits, direct spends, queued spends, insertion and exits. It does not wrap the history rings; no steady-state estimate is reported.

The batch workload submits 14 valid independent transfers plus two stale entries. Its divided gas includes the stale calldata and credit accounting. The single transfer and aggregate occur at different lifecycle states, so their difference includes storage effects. Standalone verifier measurements isolate the same verifier API across batch sizes.

Proving uses a fresh Node process per proof and the

Table 3: Standalone verifier transaction gas per proof, using actual transfer relations.

Operation	Native $k=12$	Hashed $k=1$
Single	313,968	223,126
Batch 1	349,290	253,342
Batch 2	208,007	154,504
Batch 4	137,393	105,086
Batch 8	102,087	80,394
Batch 16	84,458	68,057

Table 4: Witness calculation plus proving, seconds, and peak process RSS in MiB. Samples are lifecycle proofs, not a latency distribution.

Circuit	n	Median	Range	RSS
transfer24	17	1.10	0.88–1.18	979
transfer84	1	3.01	3.01–3.01	2059
exit	5	0.83	0.82–0.97	978
direct	2	2.34	1.72–2.96	1410
batch1	8	5.47	5.05–5.96	5129
transfer24h	17	3.31	3.26–3.54	3800
transfer84h	1	6.29	6.29–6.29	5048
exith	5	3.05	3.01–3.13	3481
directh	2	6.44	6.42–6.45	4823
batch16	6	48.35	43.52–90.63	16884

default snarkjs worker pool. Times include witness generation and proving-key loading, and exclude setup, circuit compilation, and verification. RSS includes the process and its worker threads. Workloads vary, especially insertion occupancy. These samples establish desktop feasibility; they do not establish phone or browser latency, p95, or a production profile choice.

The recorded evidence contains source and artifact digests, circuit counts, key provenance, proof timings, and successful transaction receipts. Regeneration refuses stale test evidence. No comparative gas result for Tornado Cash, Nova, or RAILGUN is asserted by this run.

ERC-20 rows use the local exact-transfer token fixture, not a mainnet token implementation. Different token code adds its own execution cost. Proof timers start inside the worker after module loading; process startup and artifact-integrity preflight are excluded.

The standalone size-16 verifier measurement uses fourteen distinct valid transfer proofs and two repetitions. Duplicate proofs are permitted by the verifier; the pool enforces replay prevention through nullifiers. The pool batch workload accepts fourteen distinct transfers and skips two stale submissions.

6.1 Transparent spend feasibility experiment

An isolated one-input, two-output relation retains the depth-32 tree, hidden asset, 128-bit amounts, ownership, commitments and nullifier rules. It compiles to 10,983 constraints. A pinned ProveKit Spartan/WHIR adapter generates and verifies actual native proofs, with Keccak, sumcheck blinding and the zero-knowledge witness-commitment mode. All 15 circuit cases and 16 native rejection checks per profile pass; both recipient ciphertexts are recovered. This experiment is not integrated into the pool.

The upstream default and twelve witness-commitment tuning profiles retain the upstream 128-bit Johnson-bound commitment target and 10-bit per-slot proof-of-work budget. The default encoded proof occupies 302,352 bytes, including required authentication hints. The smallest measured calldata floor comes from fold4-rate6: 183,728 bytes and 7,220,660 gas in a signed Prague transaction. With public inputs, ABI framing and recipient ciphertexts, the transport charge is 7,240,710 gas. These transactions perform no proof verification or payment state changes. Their charges already exceed the provisional 1,000,000-gas ceiling; they are not complete-payment gas measurements. The parameter search is bounded, and these results do not establish a lower bound for other encodings or proof systems.

The tested construction also fails a generic zero-knowledge check. For a two-row relation with identical constraints $x \cdot 1 = x$, the verifier recovers the unconstrained private witness x from a transmitted alpha evaluation, despite the witness-hiding mode. The counterexample reproduces for every tested profile. It does not establish payment-key extraction. These cost and privacy results reject the tested candidate; A Solidity verifier, insertion integration and a matching Groth16 comparison are not implemented. No qualifying transparent replacement is claimed. Source, dependency pins, test scope and generated evidence are retained under `experiments/transparent/` and `benchmarks/transparent-spend.json`.

7 Security Analysis

Balance (G1). A note enters the tree only through an insertion or direct proof. Its leaf is either a commitment output by a verified spend, whose value is conserved by the relation in Section 4.6, or a deposit commitment computed in-circuit from words bound by the SHA-256 accumulator to the amount and asset the contract collected. Forging a leaf therefore requires a SHA-256 collision at 253 bits (about 2^{126} work), a Poseidon collision, or breaking Groth16 knowledge soundness. Insertion proves each target slot empty, which rules out overwriting. Per-asset liabilities and the single-asset constraint prevent value moving between assets. The contract-side backing checks catch accounting divergence, but they are not the primary defense.

Spend authority (G2). Membership and ownership are proven from sk, and the nullifier depends on $nk = \mathsf{P}_{NK}(sk)$ and the leaf position, so a note has exactly one nullifier. Tagging the Poseidon domains prevents cross-use of hash outputs.

Statement binding. A proof must bind every externally meaningful field; otherwise a copied proof could be replayed with altered calldata. The canonical body, in-

cluding recipient, deadline, executor, ciphertexts, and in direct mode the reward beneficiary and expected queue state, is hashed to h_{ext}, a public input. h_{ext} enters a quadratic constraint in the native profile and the statement hash in the hashed profile. Groth16 is weakly simulation-extractable [22]: a re-randomized proof verifies only for the original statement, and resubmitting it repeats the original nullifiers. Tests confirm that an honest proof fails for a mutated body and for a swapped beneficiary. A formal review of the compiled constraint system remains a release requirement.

Batch verification. The soundness bound is given in Section 4.7. Skipping uses only state that other transactions can change, and it happens before verification, so an entry cannot be both skipped and applied.

Privacy (G3). A pure transfer publishes two nullifiers, four commitments, ciphertexts and a zero public asset. Commitments for different assets share one tree, but this does not establish a uniform anonymity set over every unspent note. Deposit and withdrawal histories, timing, asset availability and participants' own note knowledge can narrow the candidate set. A broadcaster paid in a shielded output learns that output's asset and value; the single-asset relation therefore reveals the transfer asset to that broadcaster. The following also leak by design:

- deposit and withdrawal amounts and assets, and their timing;
- the submitting account;
- co-submission of the transfers in one batch.

In direct mode, spending a deposit in the same call links the input to that deposit with high probability; wallets should default to waiting. Recipient address privacy depends on ristretto255 and the SHA-512 map to H_d.

Cryptographic margins. BN254 offers roughly 100-bit security after the extended tower number field sieve [23], [24], a limit shared by every system that uses Ethereum's BN254 precompiles. The Groth16 setup requires a per-circuit phase-2 ceremony, and a compromised ceremony breaks soundness for that circuit.

8 Limitations and Future Work

Transparent proof system. A replacement must cover spends, exits, direct mode, insertion and any aggregation layer without trusted setup. No qualifying replacement has been implemented. The isolated experiment in Section 6.1 fails its privacy and gas gates and does not provide a complete-payment benchmark. The current Groth16 batching equation and gas figures do not establish the cost of a replacement. Evaluation must include proof calldata, contract-size limits, batch occupancy and low-volume unilateral submission and exit costs. Aggregating transparent user proofs must preserve witness privacy and permissionless submission.

Cost floor. Two 22,100-gas nullifier writes and about 1.9 KB of calldata remain per transfer. Lowering them would require an aggregated nullifier structure proven off-chain, which reintroduces sequencing questions that the direct and batched lanes avoid.

Code size. The pool has little EIP-170 headroom. Inbox addresses, fixed-size encrypted memos and direct absorption of a pending transfer remain future work. Adding them requires a separately reviewed architecture; an extension split has not been implemented.

Evaluation scope. The measurements use actual full-depth relations with development verifying keys in one local EVM. They do not establish production setup integrity, mainnet readiness, browser or phone performance, ring-wrap steady-state gas, long-running adversarial behavior, or reorganization recovery. The permissionless asset profile and network services have not been implemented. No external audit or comparative third-party gas benchmark has been performed.

Cross-chain entry. An atomic-swap extension is specified separately, using secret revelation and a cross-group discrete-log proof [25]. The v0.8 ETH-side kernel has local tests for domain separation, credit accounting, separated claim and delivery, retry, refund and withdrawal. A pool fixture isolates those kernel tests. No Monero daemon, cross-group proof, stagenet swap or end-to-end atomicity claim is included; the extension remains inactive.

9 Conclusion

ShieldedETH v0.8 is an executable research baseline for a multi-asset payment pool on Ethereum L1. Proofs move note-tree computation out of the contract; direct insertion supports low-volume use; and batching shares the cost of verifying independent user proofs. Full-depth circuit builds, actual relation proofs, adversarial tests and complete transaction receipts support these implementation claims. Production requires a transparent proof system, independent review, broader adversarial testing and operational validation. The evaluated Groth16 backend does not satisfy the no-trusted-setup requirement.

References

[1] E. Ben-Sasson et al., "Zerocash: Decentralized anonymous payments from Bitcoin," in IEEE Symposium on Security and Privacy, 2014, pp. 459–474.

[2] D.-E. Hopwood, S. Bowe, T. Hornby, and N. Wilcox, "Zcash protocol specification," Electric Coin Company, 2024.

[3] V. Buterin, "EIP-170: Contract code size limit," Ethereum Improvement Proposals, 2016.

[4] A. Pertsev, R. Semenov, and R. Storm, "Tornado Cash privacy solution, version 1.4," 2019.

[5] Tornado Cash developers, Tornado Nova: Shielded pool with arbitrary amounts and internal transfers. (2021). Available: https://github.com/tornadocash/tornado-nova

[6] RAILGUN contributors, "RAILGUN documentation: Fees, broadcasters, and Private Proofs of Innocence." [Online]. Available: https://docs.railgun.org

[7] Aztec Labs, "Aztec network documentation." [Online]. Available: https://docs.aztec.network

[8] C. Shikhelman, M. Komarov, and A. Moskvin, "Shielded Bitcoin: Private transfers on the Bitcoin L1," Sep. 2026, [[alloc] init].

[9] V. Buterin, J. Illum, M. Nadler, F. Schär, and A. Soleimani, "Blockchain privacy and regulatory compliance: Towards a practical equilibrium," Blockchain: Research and Applications, vol. 5, no. 1, 2024.

[10] N. Gailly, M. Maller, and A. Nitulescu, "SnarkPack: Practical SNARK aggregation," in Financial Cryptography and Data Security, 2022.

[11] M. Bellare, J. A. Garay, and T. Rabin, "Fast batch verification for modular exponentiation and digital signatures," in EUROCRYPT, 1998, pp. 236–250.

[12] A. L. Ferrara, M. Green, S. Hohenberger, and M. Ø. Pedersen, "Practical short signature batch verification," in CT-RSA, 2009, pp. 309–324.

[13] J. Groth, "On the size of pairing-based non-interactive arguments," in EUROCRYPT, 2016, pp. 305–326.

[14] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, "Poseidon: A new hash function for zero-knowledge proof systems," in USENIX Security Symposium, 2021.

[15] H. de Valence, J. Grigg, M. Hamburg, I. Lovecruft, G. Tankersley, and F. Valsorda, "The ristretto255 and decaf448 groups," IETF, RFC 9496, 2023.

[16] Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF protocols," IETF, RFC 8439, 2018.

[17] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," IETF, RFC 5869, 2010.

[18] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in CRYPTO, 1986, pp. 186–194.

[19] V. Buterin and C. Reitwiessner, "EIP-197: Precompiled contracts for optimal ate pairing check on the elliptic curve alt_bn128," Ethereum Improvement Proposals, 2017.

[20] A. S. Cardozo and Z. Williamson, "EIP-1108: Reduce alt_bn128 precompile gas costs," Ethereum Improvement Proposals, 2018.

[21] A. Akhunov and M. Salem, "EIP-1153: Transient storage opcodes," Ethereum Improvement Proposals, 2018.

[22] K. Baghery, M. Kohlweiss, J. Siim, and M. Volkhov, "Another look at extraction and randomization of Groth's zk-SNARK," in Financial Cryptography and Data Security, 2021.

[23] T. Kim and R. Barbulescu, "Extended tower number field sieve: A new complexity for the medium prime case," in CRYPTO, 2016, pp. 543–571.

[24] R. Barbulescu and S. Duquesne, "Updating key size estimations for pairings," Journal of Cryptology, vol. 32, pp. 1298–1336, 2019.

[25] J. Gugger, "Bitcoin–Monero cross-chain atomic swap," IACR Cryptology ePrint Archive, 2020/1126, 2020.